



HIGH PERFORMANCE COMPUTING 101

Data Transfer & Best Practices (Module 4)

Module 4 — Lesson 2

Resource Management Best Practices



Overview

- Right-sizing requests
- Managing jobs
- Storage discipline
- Where to get help
- Course wrap-up



Right-sizing requests

- Right-sizing requests
- Managing jobs
- Storage discipline
- Where to get help
- Course wrap-up



Request only what you need

- Every CPU, memory, and walltime you request is reserved exclusively for your job
 - SLURM cannot give those resources to anyone else while your job holds them
 - this is true even if your job never touches them



Request only what you need

- Over-requesting has real costs:
 - your job waits longer, because the scheduler must find a larger block of idle resources
 - other users' jobs wait longer, because resources sit reserved but unused
 - IU's HPC systems use fair-share scheduling, so over-requesting lowers your future queue priority
- **Ask for what the job needs, not the maximum available**



Measure actual usage — `sacct` and `seff`

- Start generous, measure what the job actually used, then tighten
- After a job finishes, read its real usage with `sacct`:

```
[citrain@h2 ~]$ sacct -j 9472 --format=JobName,Elapsed,MaxRSS,NCPUS,State --units=G
```

JobName	Elapsed	MaxRSS	NCPUS	State
align.sh	01:42:18	6.20G	8	COMPLETED

- `seff <jobid>` summarizes CPU and memory efficiency in one view
- Tighten for next time:
 - set `--time` from `Elapsed` plus 20–30%
 - set `--mem` from `MaxRSS` (peak memory) plus 10–20%
- Exit code `137` = killed by the out-of-memory killer; raise `--mem`



More cores $\neq$ faster

- More cores help only if your code is written to use them:
 - MPI (Message Passing Interface), OpenMP, or embarrassingly parallel workloads benefit
 - a serial program runs on exactly one core no matter how many you request; the rest sit idle
 - confirm parallelism at runtime with ``top -u $USER``, or after the job with ``seff``



Not all software uses GPUs

- GPUs help only if your code offloads work to them:
 - PyTorch, TensorFlow, GROMACS (GPU builds) use the GPU; most plain Python, R, and bioinformatics tools do not
 - in an interactive GPU session, check live activity with ``nvidia-smi``
 - confirm a GPU was allocated with:

```
$ sacct -j <jobid> --format=JobID,AllocTRES%40,State
```
- Don't request hardware your code cannot use



Managing jobs

- Right-sizing requests
- Managing jobs
- Storage discipline
- Where to get help
- Course wrap-up



Job control commands recap

- Watch the queue:
 - `$ squeue --me` — your jobs only
 - `$ squeue -p <partition>` — all jobs in a partition (gauge how busy it is)
- Cancel:
 - `$ scancel <jobid>` — cancel one job (also ``scancel -u $USER`` for all of yours)
- Inspect and control with ``scontrol``:
 - `$ scontrol show job <jobid>` — full detail on a job
 - `$ scontrol hold <jobid>` / `$ scontrol release <jobid>` — keep a pending job from starting, then let it
 - `$ scontrol suspend <jobid>` / `$ scontrol resume <jobid>` — pause and resume a running job



Why is my job pending?

- A pending job shows state `PD`. the reason is in the `NODELIST(REASON)` column of `squeue`:
 - `Priority` — other jobs ranked higher; yours will start eventually
 - `Resources` — not enough free nodes, CPUs, or memory right now
 - `Dependency` — waiting on another job to finish
 - `QOSMaxCPUPerUser` — you've hit a per-user CPU limit
 - `ReqNodeNotAvail` — a requested node is unavailable or down
- Estimate when it will start: `$ squeue --start -j 9472`

JOBID	PARTITION	NAME	USER	ST	START_TIME	NODES
NODELIST(REASON)						
9472	gpu	train_nn	citrain	PD	2026-06-13T21:15:42	1 (Priority)



Storage discipline

- Right-sizing requests
- Managing jobs
- **Storage discipline**
- Where to get help
- Course wrap-up



Recap

- `/N/scratch`` is purged: files untouched for more than 30 days are deleted without notice
 - it is fast working space, not long-term storage
- Archive to the Scholarly Data Archive (SDA) at project milestones
- Bundle many small files (for example with ``tar``) before storing or archiving
- Clean up when you go idle: move results off scratch, delete intermediates you no longer need



Where to put what

Slate Scratch	Active job input/output, intermediate results, temporary working data
Geode Home Directory	Scripts, configs, small source code; things you want to keep but that stay small
Slate Project	Shared datasets that a research group needs across multiple jobs (request via RT Projects)
Slate	Scholarly data for data-intensive workflows and analytics running Quartz or Big Red 200.
Scholarly Data Archive (SDA)	Completed datasets and results you want to archive long-term



Package installs fill up Geode fast

- Python (pip), conda, and R all install packages into your Geode home directory by default. A single machine-learning environment with PyTorch, TensorFlow, or JAX can consume 10–20 GB. A few environments and your 100 GB quota and/or your 800,000 file limit could be completely used up.
- Redirect installs to Slate to keep your Geode home directory lean:
 - For pip (virtualenv):

```
# Create the environment on Slate instead of home
python -m venv /N/slate/$USER/envs/myproject
source /N/slate/$USER/envs/myproject/bin/activate
pip install -r requirements.txt
```



Where to get help

- Right-sizing requests
- Managing jobs
- Storage discipline
- Where to get help
- Course wrap-up



Support tiers

- **IU HPC Guide** — commands, workflows, system specs, best practices
- **IU Knowledge Base (IUKB)** — background reading, policy, extended guides
- **submit a ticket** — system errors, account issues, anything blocking your work
- **office hours** — help debugging jobs and designing workflows
- **consulting** — extended support for migration, parallelization, workflows



Contacts

- **RT Engage** — research technologies engagement support
- **Research Applications and Deep Learning (RADL)** — scientific software, compilers, programming
- **SecureMyResearch** — PHI and HIPAA compliance
- **Research Storage** — the Scholarly Data Archive (SDA), Geode
- **High Performance Systems (HPS)** — Big Red 200, Quartz
- **High Performance File Systems (HPFS)** — Slate



Filing a ticket through RT Projects

- File a ticket through RT Projects, in the queue for the team that owns the system:
<https://projects.rt.iu.edu/help/?queue=hps>
- A good ticket resolves faster because it includes:
 - the job ID (`\$SLURM\_JOB\_ID`)
 - the relevant excerpt of your output or error log
 - which system is affected, and what you already tried

The screenshot shows the 'Contact Research Technologies' form on the Indiana University RT Projects website. The form has a red header with the university logo and navigation links. The main form area contains a 'Subject*' field, a 'Message*' text area, and a 'Send' button. The footer includes the university logo, name, and accessibility/copyright information.

Indiana University

Home Center Summary Publications * Project * Help

skyangel

Contact Research Technologies

Subject*

Message*

Send

INDIANA UNIVERSITY

Accessibility | College Scorecard | Open to All | Privacy Notice | Copyright © 2026 The Trustees of Indiana University



Course wrap-up

- Right-sizing requests
- Managing jobs
- Storage discipline
- Where to get help
- Course wrap-up



Objectives met

- **Module 1** — parallelism, supercomputers, storage, how HPC is used
- **Module 2** — compute accounts, logging in, software modules, submitting and managing jobs
- **Module 3** — IU's file systems and how to use them
- **Module 4** — moving data; right-sizing and managing your resources

The ultimate goal: to become an effective, efficient, and conscientious user of IU HPC



Next steps

- Where to go next:
 - IU HPC Guide for quick references, additional documentation, and to extend these skills
- Be sure to complete the reflection for this lesson

